

Introspective Robot-Gated DAgger for VLAs

Albert Author¹ and Bernard D. Researcher²

Abstract—

I. INTRODUCTION

Vision-language-action (VLA) models have demonstrated strong capabilities across a broad range of robotic manipulation tasks. By post-training a pretrained policy on task-specific demonstrations, a robot can acquire new skills with substantially less data than would be required to train a policy from scratch. Nevertheless, collecting high-quality expert demonstrations remains expensive, particularly when deployment introduces out-of-distribution (OOD) configurations that are poorly covered by the original training data. Indiscriminately expanding the dataset to cover the long tail of deployment conditions would require considerable expert effort. An important challenge is therefore to determine when expert assistance is most valuable and selectively acquire demonstrations that improve the policy under a limited supervision budget.

Interactive imitation learning provides a natural mechanism for acquiring supervision from states encountered by the learned policy. DAgger mitigates covariate shift by querying an expert on policy-visited states and aggregating the resulting supervision into the training set. Human-gated DAgger allows a supervisor to intervene only when correction is needed, but still requires continuous human monitoring and timely control. Robot-gated DAgger automates this decision by using an uncertainty or failure signal to determine when expert assistance should be requested. However, most existing robot gates treat intervention primarily as a binary detection problem: the gate is evaluated by whether it distinguishes successful and failing executions, or whether it reduces the frequency of expert queries. The timing of the query itself, and the learning value of the demonstration produced after that query, have received considerably less attention.

This omission is important because the onset of expert control determines which states and actions enter the corrective dataset. An unnecessarily early takeover may spend expert effort reproducing a long prefix that the policy has already mastered. Conversely, an intervention made only after an observable error may collect a trajectory dominated by recovery behavior, such as retreating, regrasping, or undoing an incorrect transition. Both interventions may

successfully complete the task, and both may be considered correct by a conventional failure-detection metric, yet their demonstrations can provide substantially different supervision for subsequent policy learning. Reliable failure detection is therefore a necessary qualification for a robot gate, but it is not sufficient to determine whether the resulting data acquisition is efficient.

In this work, we formulate *learning-efficient expert takeover* for robot-gated DAgger. We view takeover timing as a data-selection decision and evaluate a takeover rule by the downstream policy improvement obtained under a fixed expert-action budget. To instantiate this idea, we propose a lightweight monitor that measures deviations in the internal representations of a VLA using an in-distribution PCA subspace. When the resulting score exceeds a calibrated threshold, the robot requests assistance and the expert takes control until task completion. The collected expert suffix is then used to fine-tune the policy together with the original demonstrations.

We further introduce an evaluation framework that separates failure-detection reliability from the learning utility of expert intervention. It combines passive failure detection, controlled comparisons of takeover timing, and matched-budget policy updates to evaluate the complete path from monitoring to data acquisition and subsequent policy learning. Across controlled manipulation tasks and multiple VLA architectures, our results show that takeover timing substantially affects downstream performance and that internal-feature gating collects more useful supervision than the evaluated baselines under the same expert budget.

Our contributions are threefold:

- We formulate *expert takeover timing* as a central problem in robot-gated DAgger, emphasizing that a robot gate determines not only whether expert assistance is requested, but also where the resulting corrective demonstration begins.
- We propose a lightweight internal-feature PCA monitor that initiates expert takeover using representations already computed by the VLA, without failure demonstrations or an auxiliary detector-training stage.
- We develop a systematic and controlled evaluation framework specifically for expert takeover timing. It combines stage-localized OOD tasks, state-matched timing comparisons, explicit expert-cost accounting, passive failure-detection evaluation, and matched-budget downstream policy learning across manipulation tasks and VLA architectures.

*This work was not supported by any organization

¹Albert Author is with Faculty of Electrical Engineering, Mathematics and Computer Science, University of Twente, 7500 AE Enschede, The Netherlands albert.author@papercept.net

²Bernard D. Researcher is with the Department of Electrical Engineering, Wright State University, Dayton, OH 45435, USA b.d.researcher@ieee.org

II. RELATED WORK

A. Active Imitation Learning and Robot-Gated DAgger

Imitation learning trains policies from expert demonstrations but remains vulnerable to distribution shifts during deployment. Active imitation learning addresses this limitation by selectively acquiring expert supervision on states visited by the learned policy. DAgger is a foundational approach that iteratively queries expert actions on learner-induced states and aggregates the resulting supervision into the training set [?]. Human-gated DAgger allows a supervisor to intervene only when correction is needed, but still requires continuous human monitoring and timely control [?]. Robot-gated DAgger instead automates the intervention decision using a learned or uncertainty-based gate. Existing methods construct such gates from safety classifiers [?], disagreement among policy ensembles [?], or combined novelty and risk estimates [?]. More recent approaches exploit signals from generative action policies: Diff-DAgger uses diffusion training loss to trigger expert intervention [?], while *Uncertainty Comes for Free* derives an uncertainty signal from the diffusion denoising process and uses the resulting intervention data for policy fine-tuning [?].

Existing work typically evaluates each robot gate as a complete system, in which the monitoring signal, decision threshold, takeover onset, intervention duration, and policy-update procedure may vary simultaneously, making it difficult to determine which aspects of the gate produce its observed performance. Instead, we identify takeover timing as a first-order factor in robot-gated DAgger because it directly determines which states and expert actions enter the corrective dataset. Our work therefore provides a more targeted and disentangled evaluation of robot gates across detection reliability, intervention quality, and downstream learning utility.

B. Runtime Monitoring as Robot-Gating Signals

Robot-gated DAgger requires an online signal that indicates when autonomous execution has become unreliable and expert assistance should be requested. Although many runtime OOD and failure detectors are not originally developed for interactive data collection, their predictions can naturally serve as the basis of a robot-gating rule. Existing monitors commonly operate on either policy inputs or outputs. Input-based methods detect deviations from the training distribution in observations or learned visual representations [?], [?], whereas output-based methods estimate uncertainty or inconsistency from policy ensembles, repeated action samples, overlapping action chunks, or generative-model losses [?], [?], [?]. A related direction monitors representations computed inside the policy; for example, SAFE trains a failure predictor on VLA hidden features using labeled successful and failed rollouts [?]. Our method similarly uses internal policy representations, but constructs an ID principal subspace from successful demonstrations without auxiliary detector optimization. We use this signal not only for passive failure detection, but also as the intervention criterion in robot-gated DAgger.

III. LEARNING-EFFICIENT EXPERT TAKEOVER FOR ROBOT-GATED DAGGER

A robot gate determines not only whether expert assistance is requested, but also the state from which the expert demonstration begins. In this section, we firstly formulate robot-gated DAgger as a learning-efficient expert takeover problem: under a fixed expert-action budget, the gate should initiate expert control from states whose demonstrations provide the greatest downstream policy improvement. We then introduce a training-free competence monitor constructed from the internal representations of the deployed policy and use it to perform introspective robot-gated data aggregation.

A. Learning-Efficient Takeover Formulation

Let π_θ denote a policy and π^* an expert. An autonomous rollout is represented as $\tau = (o_0, a_0, o_1, a_1, \dots, o_T)$, where o_t is the policy observation and a_t is the executed action. If the expert takes over at timestep t_I , the executed policy becomes

$$a_t \sim \begin{cases} \pi_\theta(\cdot | o_t), & t < t_I, \\ \pi^*(\cdot | o_t), & t \geq t_I. \end{cases} \quad (1)$$

Once intervention begins, the expert retains control until task completion. The resulting corrective demonstration is therefore the expert suffix $\tau_E(t_I) = \{(o_t, a_t^*)\}_{t=t_I}^T$.

Let $\mathcal{D}_E(t; B)$ denote the expert suffixes collected using takeover condition t under a total budget of at most B expert actions. Starting from the same base policy, these data are aggregated with the original ID demonstrations $\mathcal{D}_{ID}$ through a fixed update procedure $\text{Update}(\cdot)$. We define the learning-efficient takeover time under budget B as

$$t_B^* = \arg \max_t J_{\text{OOD}}(\text{Update}(\pi_\theta, \mathcal{D}_{ID} \cup \mathcal{D}_E(t; B))), \quad (2)$$

where J_{OOD} measures the performance of the updated policy on held-out OOD conditions.

This definition does not prescribe whether earlier or later intervention is preferable. Instead, it evaluates takeover timing by the downstream utility of the collected demonstrations under a fixed expert-action budget, without requiring manually specified failure events or timing labels.

The optimal takeover time in Eq. (2) cannot be computed online, as evaluating it requires collecting expert data, updating the policy, and measuring its downstream performance for multiple candidate takeover states. We therefore use the first departure from the policy’s ID internal-feature subspace as a deployable proxy for this optimization. Our hypothesis is that this departure marks the onset of an unsupported capability, before subsequent policy errors introduce recovery behavior. We evaluate this connection through controlled takeover-time sweeps and measure the utility regret relative to the best-performing takeover condition.

B. Training-Free Internal-Feature PCA

To obtain a deployable signal for expert takeover, we model the internal feature distribution of the VLA using principal component analysis (PCA). Let $H_i^{(\ell)} \in \mathbb{R}^{L_i \times d}$

denote the valid feature tokens extracted from an internal layer ℓ for an ID observation o_i . We aggregate them into

$$z_i^{(\ell)} = \frac{1}{L_i} \sum_{j=1}^{L_i} H_{i,j}^{(\ell)} \in \mathbb{R}^d. \quad (3)$$

The choice of layer is not fixed to a particular VLA component and is examined in Section ??.

Using features from the original ID demonstrations, we compute their empirical mean and covariance:

$$\mu_\ell = \frac{1}{N} \sum_{i=1}^N z_i^{(\ell)}, \quad \Sigma_\ell = \frac{1}{N} \sum_{i=1}^N (z_i^{(\ell)} - \mu_\ell)(z_i^{(\ell)} - \mu_\ell)^\top. \quad (4)$$

Let $V_{\ell,r}$ contain the eigenvectors corresponding to the r largest eigenvalues of Σ_ℓ . These vectors span the principal subspace of the ID internal representations.

For a deployment observation o_t , we define the Internal-Feature PCA score as its reconstruction residual outside this subspace:

$$s_{\text{PCA}}^{(\ell)}(o_t) = \left\| (I - V_{\ell,r} V_{\ell,r}^\top) (z_t^{(\ell)} - \mu_\ell) \right\|_2. \quad (5)$$

A larger score indicates a greater deviation from the internal feature distribution observed in the ID demonstrations. We calibrate the threshold γ using the $(1 - \alpha)$ quantile of the maximum scores from held-out successful ID rollouts. The resulting monitor requires neither failure data nor additional detector training.

C. Introspective Robot-Gated Data Aggregation

Introspective Robot-Gated DAgger uses the Internal-Feature PCA score to decide when expert assistance is needed. During deployment, the policy remains in control while $s_{\text{PCA}}^{(\ell)}(o_t) \leq \gamma$. Once the score exceeds γ , the robot requests assistance, and the expert takes control for the remainder of the episode.

For every successful assisted rollout, we retain the complete expert-controlled suffix and add it to the corrective dataset $\mathcal{D}_{\text{corr}}$. The policy is then fine-tuned using equal proportions of the original ID demonstrations and the collected corrective data. This procedure uses the policy’s own internal representation to select takeover states and concentrates expert supervision on regions that are poorly supported by its original training distribution.

IV. EVALUATION FRAMEWORK

A robot gate makes two coupled data-acquisition decisions. First, it determines *which policy executions require expert supervision*, thereby shaping the distribution of the collected data. Second, once assistance is requested, it determines *when expert control begins*, thereby shaping the length and behavioral content of each corrective demonstration. We design our evaluation framework to examine these two decisions both separately and jointly. We first construct controlled ID-to-OD tasks that expose data-selection and takeover-timing behavior. We then introduce complementary evaluation protocols and metrics for measuring detection

Algorithm 1 Introspective Robot-Gated DAgger

Require: ID data $\mathcal{D}_{\text{ID}}$, policy π_θ , expert π_E , PCA score s_{PCA} , threshold γ , budget B

Ensure: Updated policy π_θ

- 1: $\mathcal{D}_{\text{corr}} \leftarrow \emptyset$
- 2: $C_E \leftarrow 0$
- 3: **while** $C_E < B$ **do**
- 4: Reset environment
- 5: $\mathcal{B}_E \leftarrow \emptyset$
- 6: expert_active $\leftarrow$ false
- 7: **while** episode is not terminated **do**
- 8: **if** expert_active = false **and** $s_{\text{PCA}}(o_t) > \gamma$ **then**
- 9: expert_active $\leftarrow$ true
- 10: **end if**
- 11: **if** expert_active **then**
- 12: $a_t \sim \pi_E(\cdot \mid o_t)$
- 13: Append (o_t, a_t) to $\mathcal{B}_E$
- 14: $C_E \leftarrow C_E + 1$
- 15: **else**
- 16: $a_t \sim \pi_\theta(\cdot \mid o_t)$
- 17: **end if**
- 18: Execute a_t and observe o_{t+1}
- 19: **end while**
- 20: **if** episode succeeds and $\mathcal{B}_E \neq \emptyset$ **then**
- 21: $\mathcal{D}_{\text{corr}} \leftarrow \mathcal{D}_{\text{corr}} \cup \mathcal{B}_E$
- 22: **end if**
- 23: **end while**
- 24: $\pi_\theta \leftarrow \text{BEHAVIORCLONING}(\pi_\theta, \mathcal{D}_{\text{ID}}, \mathcal{D}_{\text{corr}})$
- 25: **return** π_θ

reliability, intervention quality, and the downstream learning utility of the collected demonstrations.

A. ID-to-OD Task Design

Existing robot-gated DAgger evaluations commonly train the base policy, collect expert interventions, and evaluate the updated policy within the same broad task distribution. Although this setting can measure whether additional supervision improves an undertrained policy, it has two limitations. First, it does not reflect a common deployment process in which a policy is initially trained on a limited data distribution and later encounters broader operating conditions. Second, it does not provide a clean capability gap: failures may arise from heterogeneous and weakly controlled deficiencies within the same distribution, making it difficult to determine whether a gate selects genuinely informative executions or whether improvement results simply from collecting more data.

We therefore separate the initial training and deployment distributions. The base policy is trained on a restricted ID distribution and is expected to perform reliably on held-out ID configurations. The OOD split preserves the task objective, robot embodiment, observation space, and action space, while changing a designated task factor that requires a capability insufficiently represented in the ID demonstra-

tions. This controlled separation makes the missing capability explicit and enables the benefit of newly collected data to be attributed to a defined generalization gap. In a mixed ID/OOD deployment stream, a reliable gate should avoid redundant interventions on already mastered ID executions and concentrate expert supervision on OOD executions that require additional capabilities.

a) Stage-Localized Distribution Shifts.: Within each task, we construct OOD conditions that primarily affect different stages of execution. The shift may alter the object pose, grasp orientation, transport direction, or target configuration. The policy can therefore remain competent during an initial task prefix and require assistance only when it reaches the affected stage. For example, an unfamiliar grasp orientation becomes relevant near the grasping stage, whereas an unseen target configuration may require assistance only after the object has already been grasped. This design prevents a gate from performing well through a single fixed intervention time. Instead, it tests whether the gate adapts its takeover decision across the temporal structure of the task and requests assistance near the stage at which the missing capability becomes relevant.

b) Task Instantiations.: We instantiate the framework using two manipulation tasks. In **STACKCUBE**, the robot must place the red cube on top of the green cube. The ID demonstrations restrict the initial relative cube positions to a narrow region, whereas the OOD split introduces broader spatial configurations that require different reaching and transport behavior. In **PICKSINGLEYCB-AIRPLANE**, the robot must grasp a toy airplane and move it to a specified three-dimensional target. The ID split presents orientations that permit a direct grasp of the narrow fuselage region, while the OOD split rotates the airplane such that the gripper must first adopt an unfamiliar orientation. In both tasks, the base policy understands the overall objective and retains reusable task skills, but successful OOD execution requires a specific capability not sufficiently covered by the ID demonstrations.

B. Evaluation Protocols and Metrics

We evaluate robot-gated DAgger through three complementary tracks. Rather than separating experimental procedures from their metrics, each track directly connects one data-acquisition decision to the corresponding evaluation criteria: passive failure detection measures *which executions* should receive supervision, controlled takeover evaluates *when* supervision should begin, and mixed ID/OOD deployment evaluates both decisions jointly in the complete learning loop.

a) Failure Detection and Data Selection.: We first evaluate each gate passively on a fixed set of policy-only rollouts. Detector predictions do not affect the executed actions, preserving the original task outcomes and enabling a controlled comparison between each monitoring signal and policy failure. All methods use disjoint ID calibration data and are evaluated on identical rollouts and random seeds. We report AUPRC as the primary metric because failure trajectories may be less frequent than successful trajectories,

together with AUROC and balanced accuracy. Failure recall and the false-alarm rate are additionally reported to distinguish missed failures from unnecessary expert queries. This track evaluates whether a gate can identify executions that require additional supervision before expert intervention or policy updating introduces further confounding factors.

b) Controlled Takeover Timing.: We next isolate the effect of takeover timing using OOD-only policy rollouts. For each initial configuration, we restore the same policy trajectory from saved environment states and trigger expert control at several predefined stages. The expert then completes the task, producing demonstrations that differ primarily in their takeover time. We fine-tune separate policies using the data collected under each timing condition while matching the total number of expert actions and keeping all optimization settings fixed. The primary metric is the resulting improvement in OOD task success, which directly measures the downstream learning utility of each takeover condition.

We additionally characterize the collected demonstrations using two intervention diagnostics. *Direct-Completion Alignment* (DCA) measures whether an expert suffix remains aligned with a nominal successful completion, rather than requiring substantial recovery behavior:

$$\text{DCA}_i = \exp \left(-\frac{D_{\text{DTW}}(\tau_i^E, \mathcal{T}_i^*)}{\sigma_{\text{nom}}} \right), \quad (6)$$

where τ_i^E is the collected expert suffix, $\mathcal{T}_i^*$ denotes nominal expert completions for the same task configuration, and σ_{nom} normalizes the natural variation among successful demonstrations. Higher DCA indicates that takeover occurs before policy errors force the expert into a recovery-heavy trajectory.

Expert-Action Savings (EAS) measures how much expert control is avoided relative to collecting a complete expert demonstration:

$$\text{EAS}_i = \max \left(0, 1 - \frac{N_i^E}{N_i^*} \right), \quad (7)$$

where N_i^E is the number of expert actions after takeover and N_i^* is the length of a complete nominal expert demonstration for the same configuration. Higher EAS indicates that the policy retains control over task stages it can already complete. We report DCA and EAS separately because they capture complementary timing errors: unnecessarily early takeover reduces EAS, whereas delayed, recovery-heavy takeover reduces DCA. We further examine whether these diagnostics are associated with downstream policy improvement across the controlled timing conditions.

c) End-to-End Robot-Gated Data Acquisition.: Finally, we evaluate the complete robot-gated DAgger loop on a deployment stream in which ID and OOD configurations appear in a controlled mixture. Each gate autonomously determines both which rollouts receive expert supervision and when expert control begins. The resulting expert data are aggregated with the original ID demonstrations, and all policies are fine-tuned from the same base checkpoint using

identical optimization settings and matched expert-action budgets.

This track jointly evaluates data selection and takeover timing. We report the composition of the queried rollouts, including the proportions of ID and OOD executions receiving expert assistance, to measure whether the gate concentrates supervision on the intended capability gaps. We also report DCA and EAS to characterize the timing and content of the collected expert segments. The final learning outcome is measured by OOD task-success improvement under the matched expert-action budget, while held-out ID success is reported to verify retention of the original policy capabilities. Across multiple supervision budgets, we additionally report the area under the success-versus-expert-action curve, which measures how efficiently each method converts expert effort into downstream policy improvement.

V. EXPERIMENTS

A. Hypotheses and Experimental Overview

Our experiments examine three hypotheses concerning the relationship between takeover timing, intervention characteristics, and downstream policy learning.

H1: Takeover-Timing Hypothesis. Under the same expert-action budget, demonstrations initiated at different stages of policy execution produce different downstream learning utility. In particular, takeover near the stage requiring an unsupported capability is expected to be more learning-efficient than either immediate takeover or failure-recovery intervention.

H2: Diagnostic-Utility Hypothesis. Across takeover conditions, Direct-Completion Alignment (DCA) and Expert-Action Savings (EAS) are positively associated with fixed-budget policy improvement, providing training-free diagnostics of intervention quality.

H3: Gating-Efficiency Hypothesis. The proposed introspective gate selects takeover states closer to the high-utility conditions identified by the controlled timing study and thereby achieves greater policy improvement than existing data-acquisition methods under the same expert-action budget.

H1 and H2 are tested jointly through a controlled takeover-timing study. H3 is tested by comparing complete online robot-gated DAgger systems. We additionally evaluate passive failure-detection performance to measure whether each gate selects policy executions that require expert supervision.

B. Simulation Setup and Baselines

We conduct experiments on the StackCube and PickSingleYCB-Airplane tasks defined in Section IV-A. Each task contains controlled OOD conditions affecting different stages of manipulation, including grasping, transport, and placement. The base policy is trained only on the corresponding ID demonstrations, while detector calibration, online data collection, and final policy evaluation use mutually disjoint task configurations and random seeds.

a) VLA Backbones.: We evaluate three VLA backbones with different policy architectures: $\pi_{0.5}$, OpenVLA, and X-VLA. Within each task-backbone pair, all methods start from the same ID-trained checkpoint and use the same observations, action horizon, expert controller, and evaluation seeds. This comparison tests whether internal-feature monitoring remains effective across different VLA representations and action-generation mechanisms.

b) Failure-Detection Baselines.: We compare Internal-Feature PCA with representative input- and output-based runtime monitors. The input-based methods include FIDeL and CRSAIL, while the output-based methods include Diff-DAgger, ACC, and STAC. All detector thresholds are calibrated using disjoint successful ID rollouts and remain fixed during evaluation.

c) Data-Acquisition Baselines.: For downstream robot-gated learning, we compare against Offline BC, Failure-Recovery DAgger, and Diff-DAgger. Offline BC collects complete expert demonstrations independently of policy execution. Failure-Recovery DAgger approximates human-gated intervention in simulation using a successful oracle: the oracle takes control only after an observable policy failure and completes the remaining task. Diff-DAgger and our method determine takeover online using their respective monitoring signals.

d) Shared Training Protocol.: All updated policies are initialized from the same ID-trained checkpoint. Newly collected expert supervision is mixed with the original ID demonstrations using the same source-balancing ratio, and all methods use identical optimization settings, training steps, and evaluation seeds. The final benchmark comparison matches the total number of expert actions rather than only the number of assisted trajectories, since different takeover times can produce expert suffixes of substantially different lengths.

C. Failure Detection and Data Selection

We first evaluate each monitoring method passively on the same policy-only rollouts. Detector predictions do not modify the executed actions, preserving the original task outcomes and allowing the monitoring signals to be compared under identical conditions. Table I reports AUPRC, our primary threshold-independent detection metric. For Internal-Feature PCA, we report the strongest internal representation available for each task-backbone pair. Missing evaluations are denoted by X.

Internal-Feature PCA achieves the highest available AUPRC on every populated task-backbone pair. The result holds across all three VLA architectures and both manipulation tasks. This supports the use of internal policy representations as a reliable signal for selecting executions that require expert supervision.

D. Controlled Takeover-Timing Study

To test H1 and H2, we isolate takeover timing from detector reliability. For each stage-localized OOD condition, we restore matched policy executions and initiate expert control

TABLE I
FAILURE-DETECTION AUPRC (%) ACROSS TASKS AND VLA BACKBONES. HIGHER IS BETTER. BOLD INDICATES THE BEST AVAILABLE RESULT IN EACH POPULATED COLUMN.

Method	Airplane			StackCube		
	$\pi_{0.5}$	OpenVLA	X-VLA	$\pi_{0.5}$	OpenVLA	X-VLA
FIDeL	89.77	88.00	53.42	X	X	97.75
CRSAIL	88.51	88.42	55.30	X	X	99.66
Diff-Dagger	X	X	55.81	X	X	95.03
ACC	73.11	X	29.41	X	X	67.37
STAC	78.54	X	27.21	X	X	55.25
Internal-Feature PCA	96.35	90.73	61.30	X	X	100.00

under four conditions. *Immediate Takeover* assigns control to the expert from the beginning of the episode. *Early Takeover* begins after a short policy prefix but before the OOD-affected stage. *Boundary Takeover* begins immediately before the first action requiring the capability missing from the ID policy. *Failure-Recovery Takeover* begins only after the policy has unsuccessfully attempted that behavior. The expert retains control until task completion in every condition.

The resulting datasets are matched by total expert-action count and used to fine-tune identical copies of the base policy. This comparison isolates takeover timing while holding the initial configurations, expert, training procedure, and evaluation seeds fixed. We report downstream OOD success rate together with DCA and EAS. We additionally measure the Spearman correlation between each intervention diagnostic and fixed-budget OOD improvement.

E. End-to-End Robot-Gated Data Acquisition

We finally evaluate the complete data-acquisition loop. ID and OOD configurations appear in a controlled deployment stream, and each gate autonomously determines which executions receive supervision and when expert control begins. The resulting data are aggregated with the original ID demonstrations and used to update the same base policy.

Table III summarizes the OOD success rates obtained from the currently completed data-acquisition runs. Internal-Feature PCA achieves the strongest available Airplane performance for all three VLA backbones. On StackCube with X-VLA, it also improves over Offline BC and Diff-Dagger, demonstrating that the internal monitoring signal can be translated into useful expert data across tasks.

The existing $\pi_{0.5}$ StackCube robot-gated result reaches an OOD success rate of 81%, but it uses an internal Deep-kNN gate rather than Internal-Feature PCA. We therefore exclude it from the PCA row in Table III. A method-consistent Internal-Feature PCA run is still required for this task-backbone pair.

F. Ablation Studies

a) *Choice of Internal Representation.*: We examine PCA monitoring at different representation families, including the visual or VLM input, intermediate VLM states, the interface preceding action generation, and states within the action-generation module. Table IV reports the strongest

PCA result within each representation family. The optimal location varies across architectures: earlier VLM features perform best for $\pi_{0.5}$ and OpenVLA on Airplane, whereas the VLM-to-action interface and early action representations perform best for X-VLA. This result motivates selecting the monitored representation using disjoint ID validation data instead of assuming that one fixed layer is optimal for every VLA architecture.

APPENDIX

Appendices should appear before the acknowledgment.

ACKNOWLEDGMENT

REFERENCES

TABLE II

CONTROLLED TAKEOVER-TIMING STUDY UNDER MATCHED EXPERT-ACTION BUDGETS. THE FOUR MIDDLE COLUMNS REPORT OOD SUCCESS RATE (%) AFTER FINE-TUNING. THE FINAL COLUMNS REPORT SPEARMAN CORRELATIONS BETWEEN INTERVENTION DIAGNOSTICS AND OOD SUCCESS IMPROVEMENT. HIGHER IS BETTER.

Task	Backbone	Immediate	Early	Boundary	Recovery	$\rho(\text{DCA}, \Delta\text{SR})$	$\rho(\text{EAS}, \Delta\text{SR})$
Airplane	$\pi_{0.5}$	X	X	X	X	X	X
Airplane	OpenVLA	X	X	X	X	X	X
Airplane	X-VLA	X	X	X	X	X	X
StackCube	$\pi_{0.5}$	X	X	X	X	X	X
StackCube	OpenVLA	X	X	X	X	X	X
StackCube	X-VLA	X	X	X	X	X	X

TABLE III

OOD SUCCESS RATE (%) AFTER POLICY FINE-TUNING. THE CURRENTLY POPULATED ENTRIES USE THE COMPLETED FIXED-TRAJECTORY-COUNT COLLECTION RUNS; THE FINAL BENCHMARK WILL MATCH TOTAL EXPERT-ACTION BUDGETS. HIGHER IS BETTER.

Method	Airplane			StackCube		
	$\pi_{0.5}$	OpenVLA	X-VLA	$\pi_{0.5}$	OpenVLA	X-VLA
Offline BC	0	30	72	49	X	58
Failure-Recovery	63	26	35	29	X	76
Diff-Dagger	74	X	58	34	X	64
Internal-Feature PCA	81	50	84	81	X	67

TABLE IV

AUPRC (%) OF INTERNAL-FEATURE PCA ACROSS REPRESENTATION FAMILIES. EACH CELL REPORTS THE STRONGEST AVAILABLE LAYER WITHIN THE CORRESPONDING FAMILY. MISSING MEASUREMENTS ARE DENOTED BY X.

Task	Backbone	Input/Encoder	VLM Hidden	VLM–Action Interface	Action Module
Airplane	$\pi_{0.5}$	96.35	95.83	91.30	90.35
Airplane	OpenVLA	90.73	87.79	X	82.78
Airplane	X-VLA	X	57.67	61.30	58.10
StackCube	X-VLA	99.53	99.81	99.94	100.00